

Allocation Dynamique de Tâches dans les Systèmes Multi-Robots

Lionel Alves Vieira, Alexis Dufossé, and Laurent Chen

Sorbonne Université

21 décembre 2025

Résumé

Cette synthèse propose une analyse des approches récentes et fondatrices du problème d'allocation dynamique de tâches dans les systèmes multi-robots, en se focalisant sur les contraintes et les incertitude.

1 Introduction

Les systèmes multi-robots connaissent aujourd'hui un essor considérable dans de nombreux domaines applicatifs, allant de la logistique automatisée à l'exploration de structures encore inconnues, en passant par la surveillance, la recherche-secours (*search and rescue*) ou encore l'assistance aux humains. Dans ces environnements, l'un des défis scientifiques clés réside dans la capacité à coordonner efficacement un ensemble d'agents robotiques autonomes pour accomplir des tâches potentiellement variées, distribuées entre les robots et dans leur environnement, tout en étant dynamique et évolutive dans le temps. Ce problème, connu sous le nom d'**allocation de tâches multi-robot** (*Multi-Robot Task Allocation, MRTA*), constitue un champ central de la robotique, notamment à cause de sa difficulté computationnelle - souvent NP-difficile - et de son importance dans ses applications.

L'objectif général du MRTA est d'assigner à chaque robot un sous-ensemble de tâches de manière à **optimiser** un critère global donné (coût, temps d'exécution et/ou de réalisation, énergie, couverture, robustesse, etc.) tout en respectant des **contraintes** inhérentes au domaine ou explicité par l'utilisateur. Une première formalisation rigoureuse des différentes variantes du problème (notamment d'où vient le terme MRTA) a été proposée par *Gerkey* et *Mataric* [?], dont la nomenclature (ST-SR / MT-MR / etc.) distingue notamment les tâches simples ou multiple, la capacité des agents à exécuter simultanément plusieurs actions, ainsi que les différents types d'allocation (unique, multiple, instantanée ou continue). Cela n'empêche pas cette taxonomie d'évoluer encore aujourd'hui, par exemple avec le terme introduit cette année par *Esikeri*, *Kyrki*, *Baumann* et *Kucner* [?] pour décrire une allocation avertie des contraintes humaines, HATA. Cette grille d'analyse reste donc même aujourd'hui une référence.

Dès la fin des années 1990, les travaux fondateurs de *Parker* et son architecture ALLIANCE [?], ont mis en

évidence l'importance d'intégrer la robustesse aux défaillances et les différences entre les agents dans les mécanismes d'MRTA. Par la suite, une approche très répandue s'est appuyée sur des principes issus de la science économique : les méthodes d'allocation par enchères, synthétisées chez *Dias*, *Stentz et al* [?]. Elles sont extrêmement répandues dans les papiers récents. Ces méthodes ont marquées un tournant dans ce domaine en introduisant des mécanismes d'optimisation distribué conciliant centralisation et décentralisation (allocation par enchères), efficacité et modularité.

Les évolutions récentes du problème quant à elle témoignent d'une gestion plus pointilleuse de la MRTA en ayant des problématiques plus complexes, souvent liées à l'**échelle**, à l'**incertitude**, ou à la **prise en compte de contraintes supplémentaires**. Plusieurs travaux contemporains se sont intéressés à un nombre beaucoup plus importants de robots [?], tandis que d'autres intègrent des contraintes temporelles [?], spatiales ou de collision [?] dans le processus d'allocation. De plus, récemment certaines approches intègrent même l'humain dans leur processus de décision [?]. Une revue complète récente du domaine [?] montre ainsi que la tendance actuelle n'est plus simplement de résoudre l'allocation des tâches, mais de répondre à des environnements dynamiques, incertains et sous contraintes.

Dans cette optique, un moyen pertinent pour synthétiser l'état de l'art consiste à structurer l'analyse selon les **types de contraintes** et les *incertitudes* auxquelles les systèmes multi-robots font face. En effet, une large partie des contributions scientifiques cherchent avant tout à répondre à la question de comment allouer des tâches efficacement lorsque le système est soumis à des contraintes temporelles, environnementales ou de mouvement, mais aussi lorsque l'environnement, la dynamique des tâches ou le comportement des autres agents (humains, robots, etc.) sont partiellement connus, probabilistes, évolutifs ou dynamiques. Cette structuration par contraintes et incertitudes permet non seulement de mettre en lumière les défis de chaque famille de problème, mais aussi d'être critique quant aux méthodologies employées pour les traiter.

Notre synthèse adoptera donc cette logique : après une présentation détaillée d'un article d'une approche de référence et de son algorithme : SCOPA [?]. Nous analyserons plusieurs approches sélectionnées par rapport à leur type de contraintes et incertitudes. L'objectif est de proposer une synthèse critique de l'état de l'art et des

méthodes existantes, afin d'identifier leur convergences et divergences et de dégager les tendance actuelles du domaine. Enfin, une conclusion mettra en évidence les limites ainsi que les questions ouvertes qui influencent la recherche actuelle sur l'allocation dynamique de tâches dans les systèmes multi-robots.

2 Approche de référence : SCoBA [?]

Dans cette synthèse, nous examinons de près l'algorithme **SCoBA** (*Stochastic Conflict-Based Allocation*). Cet algorithme reflète une évolution importante : au lieu de considérer l'allocation comme une simple correspondance entre robots et tâches à un instant donné, il traite le problème comme une planification détaillée dans le temps, avec des **contraintes de fenêtre temporelle** et une **incertitude** sur l'issue des tâches.

2.1 Architecture Hiérarchique

L'innovation principale de SCoBA réside dans son architecture qui découple deux problèmes mathématiquement complexes pour les rendre plus traitables : la planification stochastique individuelle et la coordination multi-agents.

Low-Level : Single-Agent Policy Generation Le low-level se concentre sur un agent unique, ignorant totalement la présence des autres robots. Son objectif est de générer une politique qui minimise l'espérance de pénalité de l'agent. Pour ce faire, l'algorithme construit un **Arbre de Politique** (*Policy Tree*) en balayant l'axe temporel. Cet arbre contient deux types de nœuds :

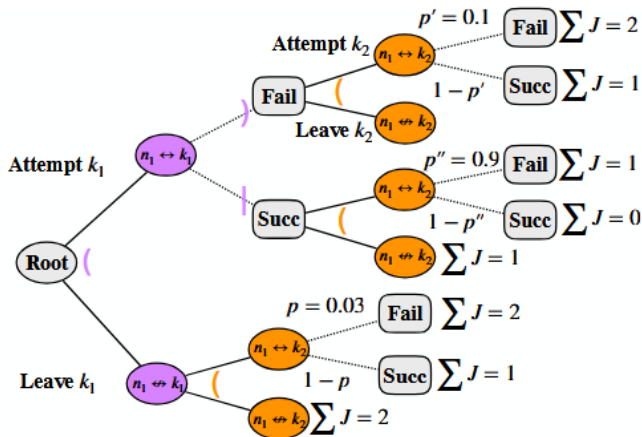


FIGURE 1 – Schéma d'un arbre de politique pour un agent n_1 et deux tâche k_1 et k_2 durant une fenêtre de temps donnée.

- **Nœuds de décision** : Au début d'une fenêtre de temps, l'agent choisit de *Tenter* ($n \leftrightarrow k$) ou de *Laisser* ($n \leftrightarrow k$) une tâche.
- **Nœuds de résultat** : Si une tâche est tentée, le résultat est probabiliste (*Succès* ou *Échec*).

L'agent utilise la programmation dynamique pour propager les valeurs des feuilles vers la racine. Pour un nœud

de résultat, la valeur est l'espérance pondérée :

$$V(\text{parent}) = p \cdot V(\text{Fail}) + (1 - p) \cdot V(\text{Succ})$$

Pour un nœud de décision, l'agent fait le choix optimal :

$$V(\text{parent}) = \min\{V(\text{child}_1), V(\text{child}_2)\}$$

Cette étape garantit que chaque agent dispose de la meilleure stratégie "égoïste" possible face à l'incertitude.

High-Level : Multi-Agent Coordination Le high-level utilise une adaptation de la méthode *Conflict-Based Search* (CBS) pour gérer les interactions. Au lieu de planifier pour l'ensemble de la flotte (ce qui exploserait en complexité), ce niveau cherche uniquement à résoudre les **conflits**. Un conflit est défini par l'intersection des plans de deux agents (par exemple A et B) sur la même tâche k durant des fenêtres de temps chevauchantes.

SCoBA construit alors un **Arbre de Contraintes** (*Constraint Tree*) :

1. Si un conflit $\{A, B\}$ sur k est détecté, le nœud courant est divisé en deux branches (scénarios).
2. **Branche Gauche** : On ajoute la contrainte "*Agent A cannot take k*". L'agent A est forcé de replanifier (via le Low-Level) sans cette tâche.
3. **Branche Droite** : On ajoute la contrainte "*Agent B cannot take k*".

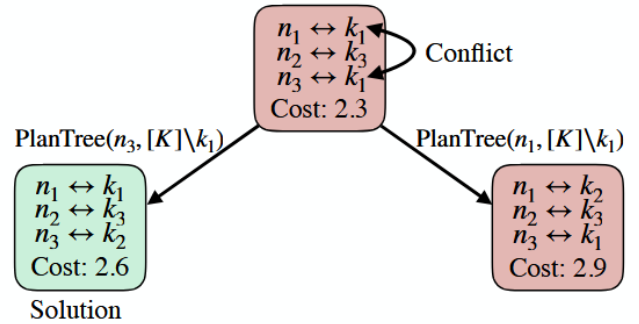


FIGURE 2 – Schéma d'un arbre de contraintes gérant un conflit entre deux agents n_1 et n_3 sur la tâche k_1 .

L'algorithme explore cet arbre avec une stratégie *Best-First Search*, garantissant de trouver la combinaison de contraintes qui minimise le coût global.

2.2 Fonction Objectif et Optimisation

L'objectif global est de minimiser le nombre espéré de tâches non réalisées (ou en retard). La force de SCoBA est que la fonction de coût globale $J(\pi)$ est simplement la somme des coûts optimaux individuels calculés par le niveau bas :

$$\min_{\pi \in \Pi} J(\pi) = \sum_n V(\text{root}_n)$$

où $V(\text{root}_n)$ est la valeur à la racine de l'arbre de politique de l'agent n . Cette propriété additive permet une communication efficace : les agents n'ont besoin de transmettre que leur coût espéré et leurs conflits potentiels au coordinateur, préservant ainsi la modularité.

2.3 Résultats Expérimentaux et Performance

La performance de SCoBA a été évaluée en simulation sur deux scénarios réalistes : un système de bras robotisés sur un tapis roulant (*Conveyor Belt*) et une flotte de drones de livraison. Les auteurs comparent leur approche à plusieurs références : des heuristiques gloutonnes (EDD, Algorithme Hongrois) et des méthodes de planification (MCTS, Q-Learning).

Les résultats, illustrés par les trois graphiques ci-dessous (issus du papier), montrent la fraction de tâches manquées (objets ratés ou livraisons en retard) en fonction de trois paramètres clés :

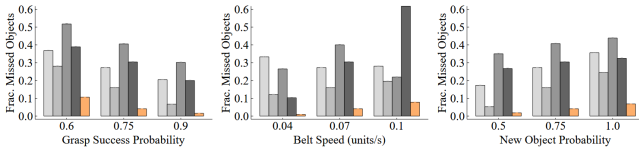


Fig. 5: Legend: □ EDD ■ Hungarian ▨ MCTS ▩ Q-Learning ■ SCoBA. On the metric of the fraction of unsuccessful tasks, i.e. objects missed, SCoBA consistently outperforms all other baselines. All results are averaged over 100 trials, with $T = 500$ time-steps per trial.

FIGURE 3 – [Image des 3 Graphiques de Résultats SCoBA : Grasp Prob, Belt Speed, Object Prob]

- 1. Probabilité de Succès (Grasp Probability) :** Même lorsque la fiabilité des robots diminue (incertitude élevée), SCoBA maintient un taux d'échec nettement inférieur aux autres méthodes, prouvant la robustesse de sa planification stochastique.
- 2. Vitesse du Tapis (Belt Speed) :** Lorsque les contraintes temporelles se durcissent (fenêtres de temps courtes), SCoBA surpasse largement les méthodes réactives qui n'anticipent pas assez.
- 3. Densité de Tâches (Object Probability) :** Face à une charge de travail élevée, SCoBA parvient à mieux répartir les tâches pour éviter la saturation, là où MCTS et Q-Learning peinent à explorer l'espace d'états.

De plus, l'algorithme démontre une scalabilité intéressante : le temps de calcul reste raisonnable pour une exécution en ligne, car la complexité est fonction du nombre de conflits réels et non du nombre total d'agents.

2.4 Analyse Critique

Forces et Garanties Contrairement aux approches heuristiques classiques (type glouton ou enchères simples), SCoBA offre des garanties théoriques fortes : il est prouvé **optimal en espérance** et **complet** (il trouvera une solution si elle existe). Sa structure hiérarchique permet une excellente scalabilité par rapport au nombre de tâches, car la complexité dépend principalement de la densité des conflits et non de la taille totale de l'espace d'états.

Limites et Hypothèses L'approche repose néanmoins sur des hypothèses structurantes :

- **Modèle d'incertitude connu :** L'algorithme suppose que la distribution de probabilité des durées ou des échecs est connue a priori ($P(\text{succès})$),

ce qui n'est pas toujours réaliste en environnement inconnu.

- **Sensibilité aux conflits :** Comme pour l'algorithme CBS dont il s'inspire, la performance peut se dégrader dans des environnements très denses (type "open space" avec peu de contraintes) où les agents ont une infinité de manières de créer des conflits, augmentant la taille de l'arbre de recherche.
- **Discrétisation :** La construction de l'arbre de politique suppose que l'on peut discrétiser les événements (début/fin de fenêtre), ce qui simplifie la réalité continue.

2.5 Conclusion sur l'approche

SCoBA représente l'état de l'art pour les problèmes dominés par des **contraintes temporelles** et une incertitude modélisable. En "interdisant" récursivement les conflits plutôt qu'en forçant les affectations, il permet au système de naviguer efficacement dans l'espace des possibles. C'est une illustration parfaite de la manière dont la contrainte (ici, l'exclusion d'une tâche) peut être utilisée comme levier d'optimisation. Cette philosophie contraste avec les approches purement réactives et servira de point de comparaison pour les méthodes focalisées sur les contraintes spatiales ou humaines présentées ci-après.

3 Approches par contraintes : Contraintes humaines [?]

L'intégration des contraintes humaines dans l'allocation de tâches multi-robots constitue aujourd'hui un enjeu notable pour les systèmes évoluant et opérant dans des environnements partagés avec des agents humains. L'objectif n'est plus seulement d'optimiser la performance, mais de garantir la sécurité et la prévisibilité des interactions humain-robot. Cette approche est souvent laissée de côté, mais le framework proposée par *Eskeri, Kyriki, Baumann et Kucner* [?], illustre particulièrement bien cette contrainte.

La solution proposée par ces chercheurs repose sur l'idée que la présence humaine impose des contraintes dynamiques qui doivent être intégrées directement dans la phase d'optimisation. Pour cela, le framework construit, appelé **HATA** pour *Human-Aware Task Allocation*, incorpore des patterns de mouvements humain directement dans le processus d'allocation.

3.1 Modélisation des contrainte humaines

La modélisation des contraintes humaines dans HATA repose sur l'usage de **Maps of Dynamics (MoDs)** : une représentation en grille de schémas probabilistes de déplacement humain dans l'environnement. Ici, on prendra l'exemple de piétons, traversant sur des chemins partagés par les agents robotiques. Ces MoDs permettent d'estimer, pour chaque zone, la **vraisemblance** qu'un humain soit rencontré dans cette zone au cours de la jour-

née. Ainsi, les préférences humaines ne sont pas définies de manière abstraite, mais dérivent directement de ces motifs de mouvements. L'information extraite des MoDs sert d'input direct à l'algorithme d'allocation.

En effet, dans HATA, ces MoDs agissent directement dans la fonction de coût utilisée pour l'allocation et pénalisent les zones à forte vraisemblance humaine. Cette fonction de coût combine deux composantes essentielles : la longueur du chemin et la **vraisemblance de croiser un humain sur ce trajet** -on ignorera ici la fonction de coût global, qui est une optimisation bayésienne de la longueur des chemins et de cette vraisemblance. Cette vraisemblance peut être écrite comme :

$$P(c_{i,j}, t + \Delta t) = \sum_{k \in K_{i,j}} \frac{T_k}{\Delta t},$$

avec $K_{i,j}$ l'ensemble des trajectoires piétonnes traversant la cellule (i, j) pendant l'intervalle $[t, t + \Delta t)$ et T_k est la durée que le piéton k passe dans la cellule (i, j) au sein de cet intervalle temporel.

3.2 Conséquences et résultats

Grâce à cette structure, le système peut réagir quasi instantanément aux changements humains. Les auteurs valident leur approche en utilisant un dataset réel de trajectoires piétonnes collectées à Osaka.

Les résultats expérimentaux sont comparés à des méthodes basées sur des plus courts chemins et des allocations à haute densité (HA-Alloc) [?] du papier montrent :

- une forte diminution des taux d'échecs et des temps de complétion ;
- une diminution des temps d'attentes ;
- et une anticipation des zones à haute densité.

Ces résultats confirment que l'ajout maîtrisé de contraintes humaines dans la fonction de coût permet d'améliorer l'efficacité des tâches. Bien que cette intégration des humains constitue déjà une avancée notable, il est probable que des modèles plus complets et précis — intégrant par exemple des modèles individuels et non des densités, ou la collaboration entre humains et robots — puissent encore améliorer l'efficacité du système.

4 Approches par contraintes : Contraintes de collision [?]

La prise en compte des **collisions** est une contrainte déjà plus étudiée dans la coordination multi-robot, notamment lorsque plusieurs agents opèrent dans un environnement très grand ou structuré. Dans de nombreuses approches classiques où que l'on a pu voir au fil de ce papier, les collisions ne sont pas traitées ou traitées en aval, au niveau de la planification des chemins / trajectoire. Cependant, une autre stratégie consiste à intégrer directement ça dans l'allocation dynamique des tâches. L'approche et le framework proposée par *Shit* et *Subudhis* [?] est un parfait exemple de cette philosophie.

4.1 Clustering de l'environnement

Le principe fondamental de leur approche repose sur l'idée que les collisions proviennent principalement d'une trop grande **proximité** spatiale ou temporelle entre robots. En découplant les zones de travail, la probabilité que deux agents se trouvent simultanément dans la même région est considérablement réduite. Concrètement, l'environnement est partitionné en plusieurs zones distinctes au moyen d'un **clustering** (K-means), chaque cluster regroupant des tâches situées dans une même région. Chaque robot est ensuite assigné à l'un de ces clusters : il devient donc le responsable de toutes les tâches situées dans cette zone géographique.

La probabilité des collisions entre un robot i et un robot j est donc formulée ainsi :

$$P(\text{collision}_{i,j}) = \int_T P(r_i \text{ at } x, t) P(r_j \text{ at } x, t) dx dt,$$

où T désigne l'horizon temporel considéré, et $P(r_i \text{ at } x, t)$ (resp. $P(r_j \text{ at } x, t)$) la probabilité que le robot r_i (resp. r_j) occupe la position x au temps t .

Une fois la zone assignée, chaque robot optimise localement son parcours. Les auteurs utilisent un algorithme de type *2-Opt* pour minimiser la longueur de la tournée à l'intérieur du cluster. Cette stratégie est efficace car chaque cluster contient relativement peu de tâches et est généralement d'une taille assez réduite. Les problèmes deviennent alors locaux et plus simples à résoudre, tout en garantissant une coordination globale.

Les résultats expérimentaux montrent que cette approche élimine presque totalement les collisions, tout en maintenant de bonnes performances en termes de temps d'exécution.

Cependant, plusieurs limites apparaissent naturellement. Le découpage spatial agit comme une forme de "distance" entre les robots plutôt que de complexifier les trajectoires avant d'éviter les collisions. Au lieu de cela, on "interdit" simplement qu'elles puissent se produire, en évitant que deux robots se croisent. On fait aussi l'hypothèse implicite que les robots sont seuls dans leur environnement, ainsi aucune possibilité de coopération entre humain-robot n'est étudiée, ce qui est assez paradoxal quand on souligne qu'un des exemples mentionnés par *Shit* et *Subudhis* sont les robots de recherche et secours.

De plus, la qualité du partitionnement dépend fortement de la structure spatiale des tâches et de l'initialisation du clustering. Aussi, ce framework suppose que l'on a à faire à des tâches homogènes et statiques, ce qui correspond davantage à des environnements industriels (dépôts, usines...) qu'à des scènes dynamiques. En bref, ce framework est donc particulièrement adaptée aux environnements **statiques** et/ou faiblement dynamiques, où les zones de travail restent cohérentes dans le temps.

4.2 Conséquences et résultats

Les résultats expérimentaux montrent que cette approche élimine presque totalement les collisions, tout en maintenant de bonnes performances en termes de temps d'exécution. L'algorithme a été comparé analytiquement à d'autres algorithmes comme le Genetic Algorithm [?] de *Shao* et la méthode gloutonne [?] de *Guo* et

Yu pour n'en citer que deux. Leur stratégie démontre que la charge de temps computationnelle peut être réduite de 93% par rapport aux autres approches, et la qualité des solutions peut aller jusqu'à être augmentée de 7% par rapport aux méthodes les plus performantes, et bien sûr, il n'y a aucune interaction entre les robots.

Malgré ces limites, cette famille d'approches présente un intérêt important : en traitant la contrainte de collision directement dans l'allocation, elle permet de simplifier considérablement la coordination et d'améliorer la planification. Des méthodes **hybrides** peuvent aussi être pensées, où une première phase de partitionnement global est complétée par des ajustements dynamiques en fonction des événements imprévus - ici, c'était les collisions. Cette direction semble particulièrement prometteuse pour les systèmes multi-robots à grande échelle.

Références

- [1] Z. Chen, M. Kumar, et al. Very large-scale multi-robot task allocation in challenging environments via robot redistribution. 2025.
- [2] Shushman Choudhury, Jayesh K Gupta, Mykel J Kochenderfer, Dorsa Sadigh, and Jeannette Bohg. Dynamic multi-robot task allocation under uncertainty and temporal constraints. In *Robotics : Science and Systems (RSS)*, 2020.
- [3] Bernard Dias, Anthony Stentz, Evan Jones, and et al. Market-based multi-robot coordination : A survey and analysis. Technical Report CMU-RI-TR-05-13, Carnegie Mellon University, 2006.
- [4] A. Eskeri, V. Kyrki, M. Baumann, and J. Kucner. Efficient human-aware task allocation for multi-robot systems in shared environments. 2025.
- [5] T. P. Kucner F. Surma and M. Mansouri. Multiple robots avoid humans to get the jobs done : An approach to human-aware task allocation. 2021.
- [6] Brian Gerkey and Maja J Mataric. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 2004.
- [7] T. Guo and J. Yu. Efficient heuristics for multi-robot path planning in crowded environments. 2023.
- [8] Lynne E Parker. Alliance : An architecture for fault-tolerant cooperative control of heterogeneous mobile robots. *IEEE Transactions on Robotics and Automation*, 1998.
- [9] Sharmila Subudhis Rathin Chandra Shit. Multi-robot task allocation for homogeneous tasks with collision avoidance via spatial clustering. 2025.
- [10] F. Rossi et al. A systematic literature review on multi-robot task allocation. 2024.
- [11] J. Shao. Robot path planning method based on genetic algorithm. 2021.